

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-210БВ-25

Студент: Заверюха Т.Д.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 27.09.26

Москва, 2026

Постановка задачи

Вариант 7.

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска вашей программы. Так же необходимо уметь продемонстрировать количество потоков, используемое вашей программой с помощью стандартных средств операционной системы. В отчете привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков. Получившиеся результаты необходимо объяснить.

Два человека играют в кости. Правила игры следующие: каждый игрок делает бросок 2-ух костей K раз; побеждает тот, кто выбросил суммарно большее количество очков. Задача программы экспериментально определить шансы на победу каждого из игроков. На вход программе подается K , какой сейчас тур, сколько очков суммарно у каждого из игроков и количество экспериментов, которые должна произвести программа

Общий метод и алгоритм решения

Использованные системные вызовы:

- `int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void*), void *arg)` – создает поток.
- `int pthread_join(pthread_t *thread, void **retval)` – ожидание завершения указанного потока
- `int clock_gettime(clockid_t clk_id, struct timespec *tp)` – получает значение времени указанных часов
- `long sysconf(int name)` – возвращает системный параметр
- `int rand_r(unsigned int *seedp)` – генератор псевдослучайных чисел

В рамках данной лабораторной работы я написал программу, которая в ходе своей работы создает потоки, распределяет потоки по ядрам в соответствии с ключом компиляции и выводит вероятность выигрыша каждого игрока. Подсчет выигрыша происходит путём вычисления количества n -ого экспериментов, которое можно задать ключом, и сложение всех выигрышных экспериментов для каждого игрока.

Назначение метрик

Для оценки эффективности многопоточной программы используются время выполнения(T), ускорение(S) и эффективность(X). Эти показатели позволяют определить, насколько применение нескольких потоков быстрее последовательного выполнения и насколько рационально используются доступные ядра процессора.

Ускорение

Ускорение показывает, во сколько раз программа с p потоками быстрее однопоточной версии:

$$S(p) = T_1 / T_p$$

где T_1 — время выполнения с одним потоком, а T_p — время выполнения с p потоками. Если $S(p) > 1$, многопоточная версия быстрее. Если $S(p) < 1$, использование потоков оказалось невыгодным.

Эффективность

Эффективность показывает, насколько полно используются потоки:

$$X(p) = S(p) / p$$

В процентах эффективность вычисляется так:

$$X(p) = S(p) / p * 100\%$$

Значение 100% соответствует идеальному масштабированию: каждый поток даёт такой же вклад, как единственный поток. На практике эффективность уменьшается из-за создания потоков, переключений контекста, планирования и конкуренции за ядра процессора.

.

Код программы

sequentialProgram.c

```
///usr/bin/cc -o /tmp/${0%%.c} -pthread $0 && exec /tmp/${0%%.c}
#include <stdint.h>
#include <stdbool.h>
#include <limits.h>

#include <stdlib.h>
#include <stdio.h>

#include <unistd.h>
#include <pthread.h>
#include <time.h>

typedef struct {
    size_t n;
    size_t k; // turn
    size_t current_turn;
    size_t start_score1;
    size_t start_score2;

    size_t start;
```

```

size_t end;

size_t wins1;
size_t wins2;
size_t draws;

unsigned seed;
size_t number;

double elapsed_sec;
} ThreadArgs;

static int roll_two_dice(unsigned *seed) {
    int a = rand_r(seed) % 6 + 1;
    int b = rand_r(seed) % 6 + 1;
    return a + b;
}

static int run_experiment(int K, int current_turn,
                        int start_score1, int start_score2,
                        unsigned *seed) {
    int sum1 = start_score1;
    int sum2 = start_score2;

    for (int t = current_turn; t <= K; ++t) {
        sum1 += roll_two_dice(seed);
        sum2 += roll_two_dice(seed);
    }

    if (sum1 > sum2) return 1;
    if (sum2 > sum1) return 2;
    return 0;
}

static void *work(void *_args)
{
    ThreadArgs *args = (ThreadArgs *)_args;

    struct timespec t0, t1;
    clock_gettime(CLOCK_MONOTONIC, &t0);

    size_t w1 = 0, w2 = 0, d = 0;

    for (int i = args->start; i < args->end; ++i) {
        int w = run_experiment(args->k, args->current_turn,
                               args->start_score1, args->start_score2,
                               &args->seed);

        if (w == 1) w1++;
        else if (w == 2) w2++;
        else d++;
    }

    args->wins1 = w1;
    args->wins2 = w2;
}

```

```

args->draws = d;

printf("thread %zu done: experiments [%zu, %zu]\n",
      args->number, args->start, args->end);

clock_gettime(CLOCK_MONOTONIC, &t1);
args->elapsed_sec = (t1.tv_sec - t0.tv_sec)
                  + (t1.tv_nsec - t0.tv_nsec) * 1e-9;

return NULL;
}

int main(int argc, char **argv)
{
    size_t K          = (argc > 1) ? (size_t)atoi(argv[1]) : 10;
    size_t current_turn = (argc > 2) ? (size_t)atoi(argv[2]) : 1;
    size_t start_score1 = (argc > 3) ? (size_t)atoi(argv[3]) : 0;
    size_t start_score2 = (argc > 4) ? (size_t)atoi(argv[4]) : 0;
    size_t N          = (argc > 5) ? (size_t)atoi(argv[5]) : 1000000;
    size_t n_threads   = (argc > 6) ? (size_t)atoi(argv[6]) :

sysconf(_SC_NPROCESSORS_ONLN - 1);

    if (current_turn < 1) current_turn = 1;
    if (current_turn > K) current_turn = K;
    if (N < 1) N = 1;

    pthread_t *threads      = malloc(n_threads * sizeof(pthread_t));
    ThreadArgs *thread_args = malloc(n_threads * sizeof(ThreadArgs));
    if (!threads || !thread_args) { perror("malloc"); return 1; }

    size_t chunk = N / n_threads;
    size_t rem    = N % n_threads;

    size_t pos = 0;
    for (size_t t = 0; t < n_threads; ++t) {
        size_t len = chunk + (t < rem ? 1 : 0);

        thread_args[t] = (ThreadArgs){
            .n          = N,
            .k          = K,
            .current_turn = current_turn,
            .start_score1 = start_score1,
            .start_score2 = start_score2,
            .start       = pos,
            .end         = pos + len,
            .wins1       = 0,
            .wins2       = 0,
            .draws       = 0,
            .seed        = (unsigned int)time(NULL) ^ (unsigned int)getpid(),
            .number      = t,
        };
        pos += len;

        int rc = pthread_create(&threads[t], NULL, work, &thread_args[t]);

```

```

    if (rc != 0) { fprintf(stderr, "pthread_create: %d\n", rc); return 1; }
}

for (size_t t = 0; t < n_threads; ++t) {
    pthread_join(threads[t], NULL);
    printf("thread %zu: %.6f s\n", t, thread_args[t].elapsed_sec);
}

size_t w1 = 0, w2 = 0, d = 0;
for (size_t t = 0; t < n_threads; ++t) {
    w1 += thread_args[t].wins1;
    w2 += thread_args[t].wins2;
    d  += thread_args[t].draws;
}

printf("K = %zu; turn = %zu; score = %zu : %zu; N = %zu;\n",
       K, current_turn, start_score1, start_score2, N);
printf("player 1 wins: %zu (%.4f)\n", w1, (double)w1 / N);
printf("player 2 wins: %zu (%.4f)\n", w2, (double)w2 / N);
printf("draws:          %zu (%.4f)\n", d, (double)d / N);

free(thread_args);
free(threads);
return 0;
}

```

parallelProgram.c

```

///usr/bin/cc -o /tmp/${0%%.c} -pthread $0 && exec /tmp/${0%%.c}
#include <stdint.h>
#include <stdbool.h>
#include <limits.h>

#include <stdlib.h>
#include <stdio.h>

#include <unistd.h>
#include <pthread.h>

typedef struct {
    size_t n;
    size_t k; // turn
    size_t current_turn;
    size_t start_score1;
    size_t start_score2;

    size_t start;
    size_t end;

    size_t wins1;
    size_t wins2;
    size_t draws;
}

```

```

    unsigned seed;
    size_t number;

    double elapsed_sec;
    size_t sims_done;
} ThreadArgs;

static int roll_two_dice(unsigned *seed) {
    int a = rand_r(seed) % 6 + 1;
    int b = rand_r(seed) % 6 + 1;
    return a + b;
}

static int run_experiment(int K, int current_turn,
                        int start_score1, int start_score2,
                        unsigned *seed) {
    int sum1 = start_score1;
    int sum2 = start_score2;

    for (int t = current_turn; t <= K; ++t) {
        sum1 += roll_two_dice(seed);
        sum2 += roll_two_dice(seed);
    }

    if (sum1 > sum2) return 1;
    if (sum2 > sum1) return 2;
    return 0;
}

static void *work(void *_args)
{
    ThreadArgs *args = (ThreadArgs *)_args;

    struct timespec t0, t1;
    clock_gettime(CLOCK_MONOTONIC, &t0);

    size_t w1 = 0, w2 = 0, d = 0;

    for (int i = args->start; i < args->end; ++i) {
        int w = run_experiment(args->k, args->current_turn,
                                args->start_score1, args->start_score2,
                                &args->seed);

        if (w == 1) w1++;
        else if (w == 2) w2++;
        else d++;
    }

    args->wins1 = w1;
    args->wins2 = w2;
    args->draws = d;

    printf("thread %zu done: experiments [%zu, %zu]\n",
           args->number, args->start, args->end);
}

```

```

clock_gettime(CLOCK_MONOTONIC, &t1);
args->elapsed_sec = (t1.tv_sec - t0.tv_sec)
                  + (t1.tv_nsec - t0.tv_nsec) * 1e-9;

return NULL;
}

int main(int argc, char **argv)
{
    size_t K          = (argc > 1) ? (size_t)atoi(argv[1]) : 10;
    size_t current_turn = (argc > 2) ? (size_t)atoi(argv[2]) : 1;
    size_t start_score1 = (argc > 3) ? (size_t)atoi(argv[3]) : 0;
    size_t start_score2 = (argc > 4) ? (size_t)atoi(argv[4]) : 0;
    size_t N            = (argc > 5) ? (size_t)atoi(argv[5]) : 1000000;
    size_t n_threads    = (argc > 6) ? (size_t)atoi(argv[6]) :

sysconf(_SC_NPROCESSORS_ONLN - 1);

    if (current_turn < 1) current_turn = 1;
    if (current_turn > K) current_turn = K;
    if (N < 1) N = 1;

    // long ncpu = sysconf(_SC_NPROCESSORS_ONLN - 1);
    // size_t n_threads = (ncpu > 1) ? (size_t)(ncpu) : 1;
    // if (N < n_threads) n_threads = N;

    pthread_t *threads = malloc(n_threads * sizeof(pthread_t));
    ThreadArgs *thread_args = malloc(n_threads * sizeof(ThreadArgs));
    if (!threads || !thread_args) { perror("malloc"); return 1; }

    size_t chunk = N / n_threads;
    size_t rem   = N % n_threads;

    size_t pos = 0;
    for (size_t t = 0; t < n_threads; ++t) {
        size_t len = chunk + (t < rem ? 1 : 0);

        thread_args[t] = (ThreadArgs){
            .n          = N,
            .k          = K,
            .current_turn = current_turn,
            .start_score1 = start_score1,
            .start_score2 = start_score2,
            .start       = pos,
            .end         = pos + len,
            .wins1       = 0,
            .wins2       = 0,
            .draws       = 0,
            .seed        = 12345u + (unsigned)t * 7919u,
            .number      = t,
        };
        pos += len;

        int rc = pthread_create(&threads[t], NULL, work, &thread_args[t]);
        if (rc != 0) { fprintf(stderr, "pthread_create: %d\n", rc); return 1; }
    }
}

```

```

}

for (size_t t = 0; t < n_threads; ++t) {
    pthread_join(threads[t], NULL);
    printf("thread %zu: %.6f s\n", t, thread_args[t].elapsed_sec);
}

size_t w1 = 0, w2 = 0, d = 0;
for (size_t t = 0; t < n_threads; ++t) {
    w1 += thread_args[t].wins1;
    w2 += thread_args[t].wins2;
    d += thread_args[t].draws;
}

printf("K = %zu; turn = %zu; score = %zu : %zu; N = %zu;\n",
       K, current_turn, start_score1, start_score2, N);
printf("player 1 wins: %zu (%.4f)\n", w1, (double)w1 / N);
printf("player 2 wins: %zu (%.4f)\n", w2, (double)w2 / N);
printf("draws: %zu (%.4f)\n", d, (double)d / N);

free(thread_args);
free(threads);
return 0;
}

```

Протокол работы программы

Число потоков	Время исполнения (мс)	Ускорение	Эффективность
1	0,214	1,00	100,0%
2	0,128	1,67	83,5%
3	0,122	1,75	58,3%
4	0,099	2,16	54,0%
16	0,106	2,02	12,6%
128	0,111	1,93	1,5%
1024	0,225	0,95	0,09%

Таблица 1 – Зависимость времени исполнения, ускорения и эффективности от числа потоков

```

barmalai@Tisha-PC:~/3sem/OS/lab2$ time ./a.out 10 1 0 0 1000000 1
thread 0 done: experiments [0, 1000000)
thread 0: 0.212591 s

```

```
K = 10; turn = 1; score = 0 : 0; N = 1000000;
```

```
player 1 wins: 481687 (0.4817)
```

```
player 2 wins: 481955 (0.4820)
```

```
draws:          36358 (0.0364)
```

```
real    0m0.214s
```

```
user    0m0.211s
```

```
sys     0m0.000s
```

```
barmalai@Tisha-PC:~/3sem/OS/lab2$ time ./a.out 10 1 0 0 1000000 1
```

```
thread 0 done: experiments [0, 1000000)
```

```
thread 0: 0.225301 s
```

```
K = 10; turn = 1; score = 0 : 0; N = 1000000;
```

```
player 1 wins: 481465 (0.4815)
```

```
player 2 wins: 481747 (0.4817)
```

```
draws:          36788 (0.0368)
```

```
real    0m0.226s
```

```
user    0m0.222s
```

```
sys     0m0.000s
```

```
barmalai@Tisha-PC:~/3sem/OS/lab2$ time ./a.out 10 1 0 0 1000000 2
```

```
thread 0 done: experiments [0, 500000)
```

```
thread 0: 0.123646 s
```

```
thread 1 done: experiments [500000, 1000000)
```

```
thread 1: 0.127017 s
```

```
K = 10; turn = 1; score = 0 : 0; N = 1000000;
```

```
player 1 wins: 481214 (0.4812)
```

```
player 2 wins: 482416 (0.4824)
```

```
draws:          36370 (0.0364)
```

```
real    0m0.128s
```

```
user    0m0.250s
```

```
sys     0m0.000s
```

```
barmalai@Tisha-PC:~/3sem/OS/lab2$ time ./a.out 10 1 0 0 1000000 3
```

```
thread 0 done: experiments [0, 333334)
```

```
thread 0: 0.097492 s
```

```
thread 1 done: experiments [333334, 666667)
thread 1: 0.107619 s
thread 2 done: experiments [666667, 1000000)
thread 2: 0.120722 s
K = 10; turn = 1; score = 0 : 0; N = 1000000;
player 1 wins: 481227 (0.4812)
player 2 wins: 481519 (0.4815)
draws:          37254 (0.0373)
```

```
real    0m0.122s
user    0m0.314s
sys     0m0.004s
```

```
barmalai@Tisha-PC:~/3sem/OS/lab2$ time ./a.out 10 1 0 0 1000000 4
```

```
thread 1 done: experiments [250000, 500000)
thread 0 done: experiments [0, 250000)
thread 0: 0.090616 s
thread 1: 0.090379 s
thread 2 done: experiments [500000, 750000)
thread 2: 0.094354 s
thread 3 done: experiments [750000, 1000000)
thread 3: 0.095610 s
K = 10; turn = 1; score = 0 : 0; N = 1000000;
player 1 wins: 480680 (0.4807)
player 2 wins: 482428 (0.4824)
draws:          36892 (0.0369)
```

```
real    0m0.099s
user    0m0.362s
sys     0m0.004ssum = 100.00 (numbers: 1)
```

```
barmalai@Tisha-PC:~/3sem/OS/lab2$ time ./a.out 10 1 0 0 1000000 16
```

```
K = 10; turn = 1; score = 0 : 0; N = 1000000;
player 1 wins: 482496 (0.4825)
player 2 wins: 479888 (0.4799)
draws:          37616 (0.0376)
```

```
real    0m0.106s
user    0m0.343s
sys     0m0.016s
```

```
barmalai@Tisha-PC:~/3sem/OS/lab2$ time ./a.out 10 1 0 0 1000000 128
```

```
K = 10; turn = 1; score = 0 : 0; N = 1000000;
```

```
player 1 wins: 493440 (0.4934)
```

```
player 2 wins: 470464 (0.4705)
```

```
draws:          36096 (0.0361)
```

```
real    0m0.111s
```

```
user    0m0.238s
```

```
sys     0m0.007s
```

```
barmalai@Tisha-PC:~/3sem/OS/lab2$ time ./a.out 10 1 0 0 1000000 1024
```

```
K = 10; turn = 1; score = 0 : 0; N = 1000000;
```

```
player 1 wins: 475712 (0.4757)
```

```
player 2 wins: 487424 (0.4874)
```

```
draws:          36864 (0.0369)
```

```
real    0m0.225s
```

```
user    0m0.272s
```

```
sys     0m0.119s
```

Вывод

В ходе лабораторной работы я приобрел практические навыки по управлению и распределению между ядер потоков в ОС Linux. При 1024 потоках на 4-ядерной системе ускорение составило ~0,95, то есть программа работает медленнее, чем последовательная версия. Это объясняется тем, что одновременно работают только 4 потока, а остальные 1020 конкурируют за процессорное время. Накладные расходы создание потоков и переключение контекста (sys = 0,119с) превышают выигрыш от параллелизма. Теоретический предел ускорения на 4 ядрах – 4, практически он достигается при числе потоков, равном числу ядер.